

Министерство науки и высшего образования Российской Федерации
Федеральное государственное бюджетное образовательное учреждение
высшего образования
«Ярославский государственный технический университет»
Кафедра «Информационные системы и технологии»

Отчет по лабораторной
работе защищен
с оценкой _____
Преподаватель
_____ В. С. Пашичев
«__» _____ 2026

Веб-сервис для салона красоты «Beauty Salon»

Отчет по лабораторной работе по курсу
«Системы управления базами данных»

ЯГТУ 09.03.02-000 ПР

Отчет выполнили
студентки группы ЦИС-28
А.С.Тарасова
В.С.Лебедева
К.О.Завадская
«__» _____ 2026

1.Цель работы

Разработать веб-сервис для салона красоты, реализующий управление записью клиентов, сотрудниками и услугами, с разграничением прав доступа и автоматизацией основных бизнес-процессов салона.

2. Актуальность (почему Google стоит купить этот проект)

Каждый день тысячи салонов красоты теряют клиентов из-за того, что запись ведётся в WhatsApp и Excel. Клиент не может записаться ночью. Администратор забывает подтвердить. Мастер приходит на пустое рабочее место.

«Beauty Salon» решает это за 5 минут установки. Клиент записывается онлайн 24/7. Мастер видит расписание в телефоне. Администратор — отчёт по загрузке.

Google получает готовый B2B SaaS-продукт, который можно интегрировать с Google Calendar и Google Workspace. Рынок — миллионы салонов по всему миру. Подписка от \$50 в месяц. Окупаемость — месяцы, а не годы.

3. Техническое задание

3.1. Назначение разработки

Веб-сервис «Beauty Salon» предназначен для автоматизации записи клиентов в салон красоты, управления сотрудниками, услугами и расписанием. Система заменяет ручную запись через мессенджеры и Excel.

3.2. Пользователи и роли

В системе предусмотрены три роли: System Administrator (полный доступ ко всем функциям), Salon Administrator (управление клиентами, услугами, записью и расписанием) и Cosmetologist (только просмотр своего расписания и записей клиентов). Аутентификация выполняется по email и паролю с выдачей JWT-токена.

3.3. Функциональные требования

Приложение обеспечивает создание, просмотр, редактирование и удаление следующих сущностей: клиенты, сотрудники, услуги, записи, расписание, роли и назначение ролей сотрудникам. Запись клиента содержит дату, время, статус (Scheduled, Confirmed, Completed, Cancelled) и привязку к клиенту, сотруднику и услуге. Расписание сотрудника включает дату, время начала и окончания работы, а также перерывы.

3.4. Технологические требования

Серверная часть разработана на Java 26 с использованием Spring Boot 4.0.6. Система сборки — Gradle. База данных — PostgreSQL, доступ через Spring Data JPA. Миграции выполняются Liquibase. Безопасность реализована на Spring Security с JWT. Код покрыт unit- и интеграционными тестами не менее чем на 70%. Настроено логирование и подключен Spring Boot Actuator.

4. Сущности базы данных

В приложении реализованы следующие сущности:

Client (Клиент). Хранит информацию о клиентах салона: фамилия, имя, отчество, телефон, email, дата рождения, дата регистрации. Связан с записью (Appointment) — один клиент может иметь много записей.

Employee (Сотрудник). Хранит данные о сотрудниках: фамилия, имя, отчество, специализация, телефон, email. Связан с записью (один сотрудник может вести много записей), с расписанием (один сотрудник может иметь много записей в расписании) и с назначением ролей (один сотрудник может иметь несколько ролей в разные даты).

Role (Роль). Хранит роли: System Administrator, Salon Administrator, Cosmetologist. Связана с назначением ролей (одна роль может быть назначена многим сотрудникам).

EmployeeRole (Назначение роли сотруднику). Связующая таблица между сотрудником и ролью. Дополнительно хранит дату назначения. Обеспечивает связь многие-ко-многим между Employee и Role.

Procedure (Услуга). Хранит услуги салона: название, стоимость, длительность. Связана с записью (одна услуга может быть в многих записях).

Appointment (Запись). Хранит записи клиентов: дата, время, статус (Scheduled, Confirmed, Completed, Cancelled), заметка. Связана с клиентом (многие записи принадлежат одному клиенту), с сотрудником (многие записи ведутся одним сотрудником) и с услугой (многие записи содержат одну услугу). Это центральная сущность, объединяющая клиента, сотрудника и услугу.

Schedule (Расписание). Хранит расписание работы сотрудников: дата, время начала, время окончания, начало перерыва, конец перерыва. Связана с сотрудником (один сотрудник может иметь много записей в расписании на разные дни).

User (Пользователь для Spring Security). Реализует интерфейс UserDetails. Хранит id сотрудника, email, пароль и роль. Используется для аутентификации и авторизации.

Связи между сущностями:

Client (1) → (M) Appointment

Employee (1) → (M) Appointment

Employee (1) → (M) Schedule

Employee (1) → (M) EmployeeRole

Role (1) → (M) EmployeeRole

Procedure (1) → (M) Appointment

Связь между Employee и Role реализована через промежуточную таблицу EmployeeRole (многие-ко-многим с дополнительным полем assignment_date).

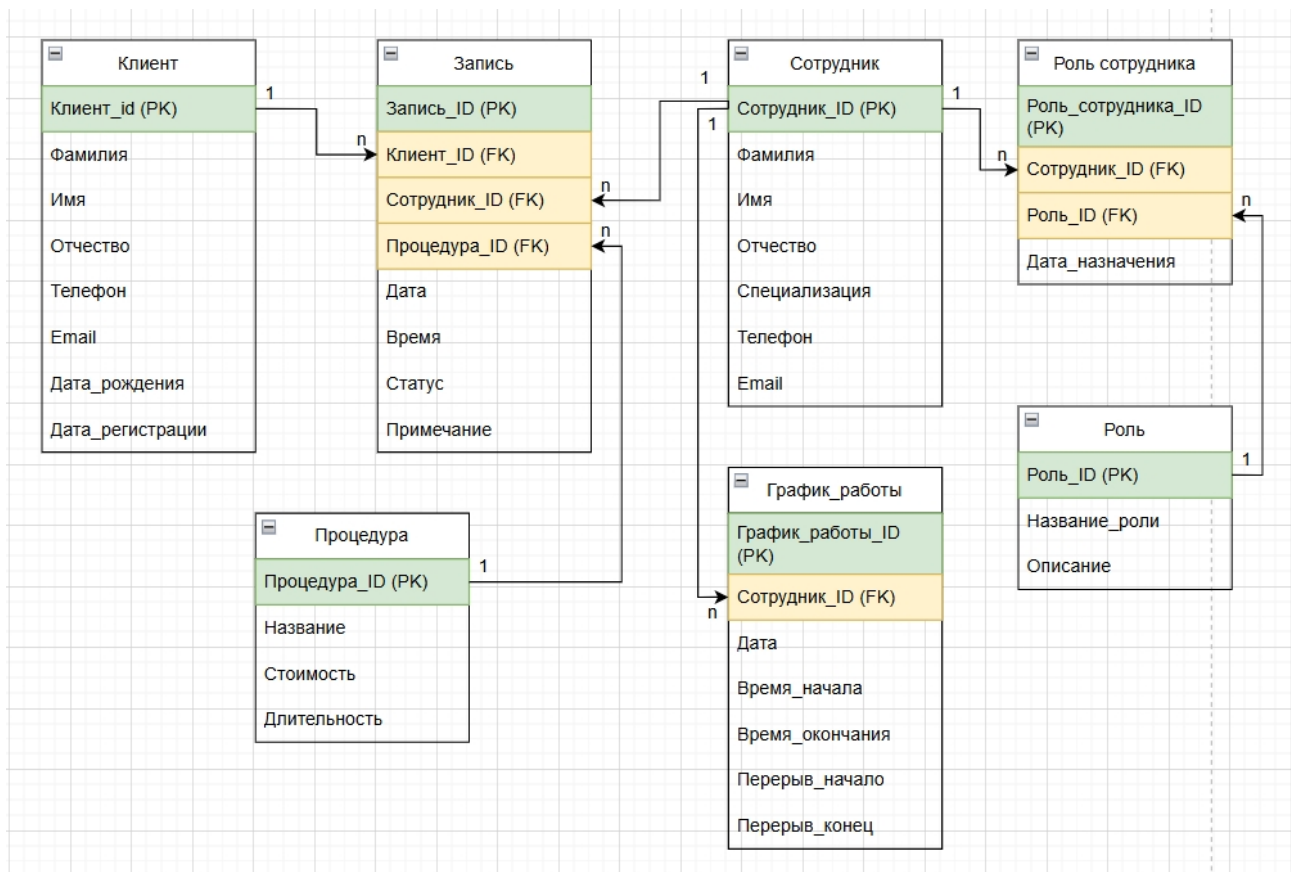


Рисунок 1-Сущности и связи между ними

5. CRUD контроллеры

В приложении реализованы REST-контроллеры для всех сущностей. Каждый контроллер предоставляет эндпоинты для выполнения операций создания, чтения, обновления и удаления (CRUD). Ниже приведён полный разбор контроллера для работы с клиентами, а также листинги остальных контроллеров.

5.1. Контроллер управления клиентами (ClientController)

Контроллер ClientController обрабатывает запросы по маршруту `/api/clients` и предоставляет следующие эндпоинты:

- GET `/api/clients` — получение списка всех клиентов
- GET `/api/clients/{id}` — получение клиента по идентификатору
- POST `/api/clients` — создание нового клиента
- PUT `/api/clients/{id}` — обновление данных клиента

- DELETE /api/clients/{id} — удаление клиента

Листинг контроллера:

```
package com.beauty.salon.controller;

import com.beauty.salon.dto.ClientCreateUpdateDTO;
import com.beauty.salon.dto.ClientDTO;
import com.beauty.salon.service.ClientService;
import jakarta.validation.Valid;
import lombok.RequiredArgsConstructor;
import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;

import java.util.List;

@RestController
@RequestMapping("/api/clients")
@RequiredArgsConstructor
public class ClientController {

    private final ClientService clientService;

    @GetMapping
    public ResponseEntity<List<ClientDTO>> getAllClients() {
        return ResponseEntity.ok(clientService.getAllClients());
    }

    @GetMapping("/{id}")
```

```

    public ResponseEntity<ClientDTO> getClientById(@PathVariable Integer
id) {

        return ResponseEntity.ok(clientService.getClientById(id));

    }

    @PostMapping

    public ResponseEntity<ClientDTO> createClient(@Valid @RequestBody
ClientCreateUpdatedDTO createdDTO) {

        ClientDTO created = clientService.createClient(createdDTO);

        return new ResponseEntity<>(created, HttpStatus.CREATED);

    }

    @PutMapping("/{id}")

    public ResponseEntity<ClientDTO> updateClient(

        @PathVariable Integer id,

        @Valid @RequestBody ClientCreateUpdatedDTO updatedDTO) {

        return ResponseEntity.ok(clientService.updateClient(id,
updatedDTO));

    }

    @DeleteMapping("/{id}")

    public ResponseEntity<Void> deleteClient(@PathVariable Integer id) {

        clientService.deleteClient(id);

        return ResponseEntity.noContent().build();

    }

}

```

Демонстрация работы эндпоинтов

Мы зашли под системным админом.

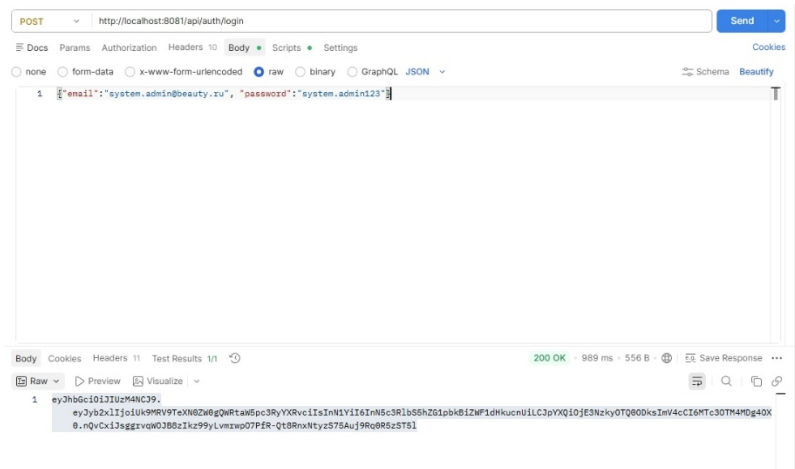


Рисунок 2-Вход под системным администратором

1. Получение списка клиентов (GET /api/clients)

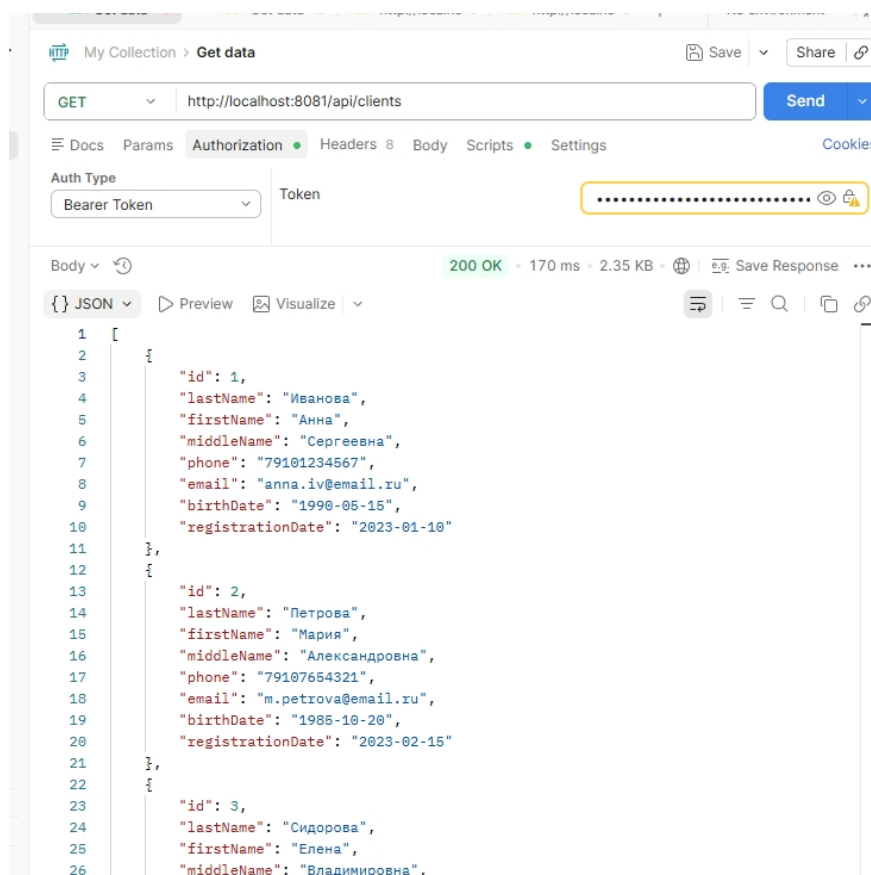


Рисунок 3-Получение списка клиентов

Данные в базе данных (таблица clients):

clients Введите SQL выражение чтобы отфильтровать результаты								
Таблица	id	last_name	first_name	middle_name	phone	email	birth_date	registration_date
1	1	Иванова	Анна	Сергеевна	79101234567	anna.iv@email.ru	1990-05-15	2023-01-10
2	2	Петрова	Мария	Александровна	79107654321	m.petrova@email.ru	1985-10-20	2023-02-15
3	3	Сидорова	Елена	Владимировна	79103451234	elena.s@email.ru	1995-03-01	2023-03-05
4	4	Козлова	Ольга	Дмитриевна	79107895678	olga.koz@email.ru	1988-12-12	2023-04-20
5	5	Морозова	Татьяна	Игоревна	79109874567	t.moroz@email.ru	1992-07-30	2023-05-12
6	6	Волкова	Наталья	Павловна	79105671234	n.volk@email.ru	1980-01-25	2024-01-25
7	7	Соколова	Ирина	Алексеевна	79102349876	i.sokol@email.ru	1998-09-14	2024-02-01
8	8	Лебедева	Светлана	Викторовна	79104562345	svet.l@email.ru	1975-11-03	2024-02-10
9	9	Павлова	Юлия	Андреевна	79106783456	j.pavlova@email.ru	2000-06-18	2024-03-01
10	10	Новикова	Анастасия	Романовна	79108904567	a.novik@email.ru	1993-04-22	2024-03-15

Рисунок 4-Данные в базе данных

2. Создание нового клиента (POST /api/clients)

Запрос в Postman (тело запроса с данными нового клиента) и полученный ответ:

POST http://localhost:8081/api/clients

Docs Params Authorization Headers 11 Body Scripts Settings

none form-data x-www-form-urlencoded raw binary

```

1 {
2   "lastName": "Петрова",
3   "firstName": "Екатерина",
4   "middleName": "Дмитриевна",
5   "phone": "79161234567",
6   "email": "ekaterina@mail.ru",
7   "birthDate": "1995-03-20"
8 }

```

Body Cookies Headers 11 Test Results

{ } JSON Preview Visualize

```

1 {
2   "id": 118,
3   "lastName": "Петрова",
4   "firstName": "Екатерина",
5   "middleName": "Дмитриевна",
6   "phone": "79161234567",
7   "email": "ekaterina@mail.ru",
8   "birthDate": "1995-03-20",
9   "registrationDate": "2026-05-20"
10 }

```

Рисунок 5-Создание нового клиента

Состояние базы данных после выполнения запроса (появилась новая запись):

clients Введите SQL выражение чтобы отфильтровать результаты								
	123 id	A-Z last_name	A-Z first_name	A-Z middle_name	A-Z phone	A-Z email	birth_date	registration_date
1	1	Иванова	Анна	Сергеевна	79101234567	anna.iv@email.ru	1990-05-15	2023-01-10
2	2	Петрова	Мария	Александровна	79107654321	m.petrova@email.ru	1985-10-20	2023-02-15
3	3	Сидорова	Елена	Владимировна	79103451234	elena.s@email.ru	1995-03-01	2023-03-05
4	4	Козлова	Ольга	Дмитриевна	79107895678	olga.koz@email.ru	1988-12-12	2023-04-20
5	5	Морозова	Татьяна	Игоревна	79109874567	t.moroz@email.ru	1992-07-30	2023-05-12
6	6	Волкова	Наталья	Павловна	79105671234	n.volk@email.ru	1980-01-25	2024-01-25
7	7	Соколова	Ирина	Алексеевна	79102349876	i.sokol@email.ru	1998-09-14	2024-02-01
8	8	Лебедева	Светлана	Викторовна	79104562345	svet.l@email.ru	1975-11-03	2024-02-10
9	9	Павлова	Юлия	Андреевна	79106783456	j.pavlova@email.ru	2000-06-18	2024-03-01
10	10	Новикова	Анастасия	Романовна	79108904567	a.novik@email.ru	1993-04-22	2024-03-15
11	118	Петрова	Екатерина	Дмитриевна	79161234567	ekaterina@mail.ru	1995-03-20	2026-05-20

Рисунок 6-Состояние базы данных после выполнения запроса

3. Обновление данных клиента (PUT /api/clients/{id})

Запрос в Postman (обновлённые данные-смена фамилии) и ответ сервера:

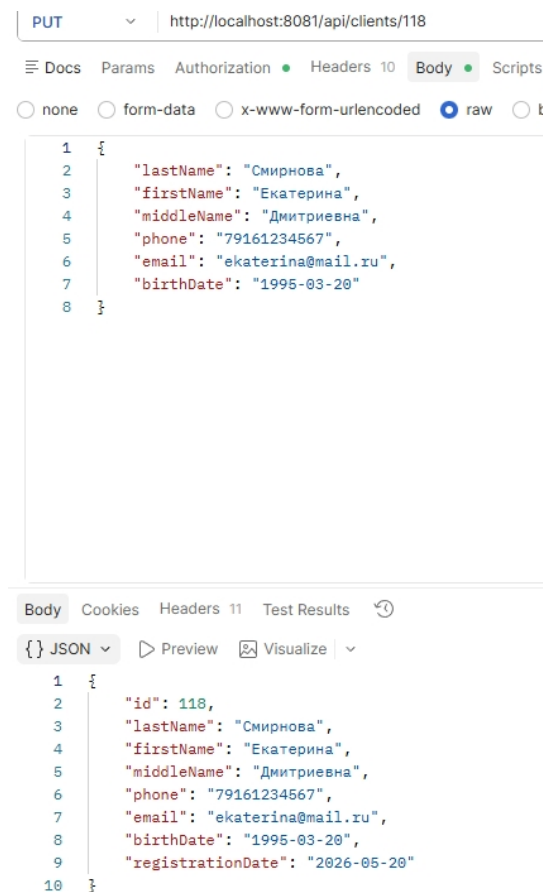


Рисунок 7-Обновление данных клиента

Состояние базы данных после обновления (данные изменены):

	id	A-Z last_name	A-Z first_name	A-Z middle_name	A-Z phone	A-Z email	birth_date	registration_date
1	1	Иванова	Анна	Сергеевна	79101234567	anna.iv@email.ru	1990-05-15	2023-01-10
2	2	Петрова	Мария	Александровна	79107654321	m.petrova@email.ru	1985-10-20	2023-02-15
3	3	Сидорова	Елена	Владимировна	79103451234	elena.s@email.ru	1995-03-01	2023-03-05
4	4	Козлова	Ольга	Дмитриевна	79107895678	olga.koz@email.ru	1988-12-12	2023-04-20
5	5	Морозова	Татьяна	Игоревна	79109874567	t.moroz@email.ru	1992-07-30	2023-05-12
6	6	Волкова	Наталья	Павловна	79105671234	n.volk@email.ru	1980-01-25	2024-01-25
7	7	Соколова	Ирина	Алексеевна	79102349876	i.sokol@email.ru	1998-09-14	2024-02-01
8	8	Лебедева	Светлана	Викторовна	79104562345	svet.l@email.ru	1975-11-03	2024-02-10
9	9	Павлова	Юлия	Андреевна	79106783456	j.pavlova@email.ru	2000-06-18	2024-03-01
10	10	Новикова	Анастасия	Романовна	79108904567	a.novik@email.ru	1993-04-22	2024-03-15
11	118	Смирнова	Екатерина	Дмитриевна	79161234567	ekaterina@mail.ru	1995-03-20	2026-05-20

Рисунок 8-Состояние базы данных после обновления

4. Удаление клиента (DELETE /api/clients/{id})

Запрос в Postman:

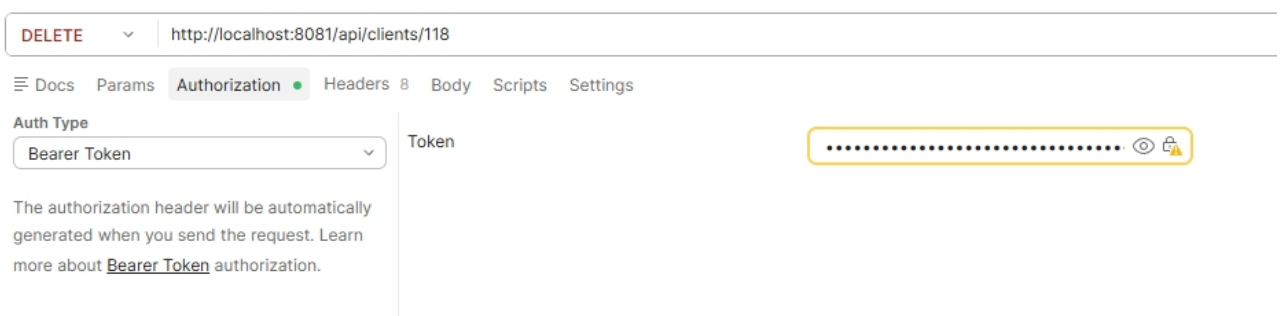


Рисунок 9-Удаление клиента

Состояние базы данных после удаления (запись отсутствует):

	id	A-Z last_name	A-Z first_name	A-Z middle_name	A-Z phone	A-Z email	birth_date	registration_date
1	1	Иванова	Анна	Сергеевна	79101234567	anna.iv@email.ru	1990-05-15	2023-01-10
2	2	Петрова	Мария	Александровна	79107654321	m.petrova@email.ru	1985-10-20	2023-02-15
3	3	Сидорова	Елена	Владимировна	79103451234	elena.s@email.ru	1995-03-01	2023-03-05
4	4	Козлова	Ольга	Дмитриевна	79107895678	olga.koz@email.ru	1988-12-12	2023-04-20
5	5	Морозова	Татьяна	Игоревна	79109874567	t.moroz@email.ru	1992-07-30	2023-05-12
6	6	Волкова	Наталья	Павловна	79105671234	n.volk@email.ru	1980-01-25	2024-01-25
7	7	Соколова	Ирина	Алексеевна	79102349876	i.sokol@email.ru	1998-09-14	2024-02-01
8	8	Лебедева	Светлана	Викторовна	79104562345	svet.l@email.ru	1975-11-03	2024-02-10
9	9	Павлова	Юлия	Андреевна	79106783456	j.pavlova@email.ru	2000-06-18	2024-03-01
10	10	Новикова	Анастасия	Романовна	79108904567	a.novik@email.ru	1993-04-22	2024-03-15

Рисунок 10-Состояние базы данных после удаления

5.2. Контроллер управления сотрудниками (EmployeeController)

Эндпоинты: `/api/employees` — получение списка, получение по ID, создание, обновление, удаление.

Листинг EmployeeController.java:

```

java

package com.beauty.salon.controller;

import com.beauty.salon.dto.EmployeeCreateUpdateDTO;
import com.beauty.salon.dto.EmployeeDTO;
import com.beauty.salon.service.EmployeeService;
import jakarta.validation.Valid;
import lombok.RequiredArgsConstructor;
import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;

import java.util.List;

@RestController
@RequestMapping("/api/employees")
@RequiredArgsConstructor
public class EmployeeController {

    private final EmployeeService employeeService;

    @GetMapping
    public ResponseEntity<List<EmployeeDTO>> getAll() {
        return ResponseEntity.ok(employeeService.getAllEmployees());
    }

    @GetMapping("/{id}")

```

```

    public ResponseEntity<EmployeeDTO> getById(@PathVariable Integer id)
{
    return ResponseEntity.ok(employeeService.getEmployeeById(id));
}

@PostMapping
    public ResponseEntity<EmployeeDTO> create(@Valid @RequestBody
EmployeeCreateUpdatedDTO dto) {
    return new ResponseEntity<>(employeeService.createEmployee(dto),
HttpStatus.CREATED);
}

@PutMapping("/{id}")
    public ResponseEntity<EmployeeDTO> update(@PathVariable Integer id,
@Valid @RequestBody EmployeeCreateUpdatedDTO dto) {
    return ResponseEntity.ok(employeeService.updateEmployee(id,
dto));
}

@DeleteMapping("/{id}")
    public ResponseEntity<Void> delete(@PathVariable Integer id) {
    employeeService.deleteEmployee(id);
    return ResponseEntity.noContent().build();
}
}

```

5.3. Контроллер управления услугами (ProcedureController)

Эндпоинты: /api/procedures — получение списка, получение по ID, создание, обновление, удаление.

Листинг ProcedureController.java:

```

java
package com.beauty.salon.controller;

import com.beauty.salon.dto.ProcedureCreateUpdateDTO;
import com.beauty.salon.dto.ProcedureDTO;
import com.beauty.salon.service.ProcedureService;
import jakarta.validation.Valid;
import lombok.RequiredArgsConstructor;
import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;

import java.util.List;

@RestController
@RequestMapping("/api/procedures")
@RequiredArgsConstructor
public class ProcedureController {

    private final ProcedureService procedureService;

    @GetMapping
    public ResponseEntity<List<ProcedureDTO>> getAll() {
        return ResponseEntity.ok(procedureService.getAllProcedures());
    }

    @GetMapping("/{id}")

```

```

    public ResponseEntity<ProcedureDTO> getById(@PathVariable Integer id)
    {
        return ResponseEntity.ok(procedureService.getProcedureById(id));
    }

    @PostMapping
    public ResponseEntity<ProcedureDTO> create(@Valid @RequestBody
    ProcedureCreateUpdateDTO dto) {
        return new
    ResponseEntity<>(procedureService.createProcedure(dto),
    HttpStatus.CREATED);
    }

    @PutMapping("/{id}")
    public ResponseEntity<ProcedureDTO> update(@PathVariable Integer id,
    @Valid @RequestBody ProcedureCreateUpdateDTO dto) {
        return ResponseEntity.ok(procedureService.updateProcedure(id,
    dto));
    }

    @DeleteMapping("/{id}")
    public ResponseEntity<Void> delete(@PathVariable Integer id) {
        procedureService.deleteProcedure(id);
        return ResponseEntity.noContent().build();
    }
}

```

5.4. Контроллер управления записями (AppointmentController)

Эндпоинты: /api/appointments — получение списка, получение по ID, получение записей сотрудника, создание, обновление, удаление.

Листинг AppointmentController.java:

```
java
package com.beauty.salon.controller;

import com.beauty.salon.dto.AppointmentCreateUpdatedDTO;
import com.beauty.salon.dto.AppointmentDTO;
import com.beauty.salon.service.AppointmentService;
import jakarta.validation.Valid;
import lombok.RequiredArgsConstructor;
import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;

import java.util.List;

@RestController
@RequestMapping("/api/appointments")
@RequiredArgsConstructor
public class AppointmentController {

    private final AppointmentService appointmentService;

    @GetMapping
    public ResponseEntity<List<AppointmentDTO>> getAll() {
        return
        ResponseEntity.ok(appointmentService.getAllAppointments());
    }
}
```



```

    @GetMapping("/{id}")

    public ResponseEntity<AppointmentDTO> getById(@PathVariable Integer
id) {

        return
ResponseEntity.ok(appointmentService.getAppointmentsById(id));

    }


    @GetMapping("/employee/{employeeId}")

    public ResponseEntity<List<AppointmentDTO>>
getByEmployeeId(@PathVariable Integer employeeId) {

        return
ResponseEntity.ok(appointmentService.getAppointmentsByEmployeeId(employee
Id));

    }


    @PostMapping

    public ResponseEntity<AppointmentDTO> create(@Valid @RequestBody
AppointmentCreateUpdatedDTO dto) {

        return new
ResponseEntity<>(appointmentService.createAppointment(dto),
HttpStatus.CREATED);

    }


    @PutMapping("/{id}")

    public ResponseEntity<AppointmentDTO> update(@PathVariable Integer
id, @Valid @RequestBody AppointmentCreateUpdatedDTO dto) {

        return ResponseEntity.ok(appointmentService.updateAppointment(id,
dto));

    }


    @DeleteMapping("/{id}")

```

```

        public ResponseEntity<Void> delete(@PathVariable Integer id) {
            appointmentService.deleteAppointment(id);
            return ResponseEntity.noContent().build();
        }
    }
}

```

5.5. Контроллер управления расписанием (ScheduleController)

Эндпоинты: /api/schedules — получение списка, получение по ID, создание, обновление, удаление.

Листинг ScheduleController.java:

```

java
package com.beauty.salon.controller;

import com.beauty.salon.dto.ScheduleCreateUpdatedDTO;
import com.beauty.salon.dto.ScheduledDTO;
import com.beauty.salon.service.ScheduleService;
import jakarta.validation.Valid;
import lombok.RequiredArgsConstructor;
import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;

import java.util.List;

@RestController
@RequestMapping("/api/schedules")
@RequiredArgsConstructor
public class ScheduleController {

```

```

private final ScheduleService scheduleService;

@GetMapping
public ResponseEntity<List<ScheduleDTO>> getAll() {
    return ResponseEntity.ok(scheduleService.getAllSchedules());
}

@GetMapping("/{id}")
public ResponseEntity<ScheduleDTO> getById(@PathVariable Integer id)
{
    return ResponseEntity.ok(scheduleService.getScheduleById(id));
}

@PostMapping
public ResponseEntity<ScheduleDTO> create(@Valid @RequestBody
ScheduleCreateUpdateDTO dto) {
    return new ResponseEntity<>(scheduleService.createSchedule(dto),
HttpStatus.CREATED);
}

@PutMapping("/{id}")
public ResponseEntity<ScheduleDTO> update(@PathVariable Integer id,
@Valid @RequestBody ScheduleCreateUpdateDTO dto) {
    return ResponseEntity.ok(scheduleService.updateSchedule(id,
dto));
}

@DeleteMapping("/{id}")
public ResponseEntity<Void> delete(@PathVariable Integer id) {

```

```

        scheduleService.deleteSchedule(id);

        return ResponseEntity.noContent().build();
    }
}

```

5.6. Контроллер авторизации (AuthController)

Эндпоинты: POST /api/auth/login — вход в систему, выдача JWT-токена.

Листинг AuthController.java:

```

java
package com.beauty.salon.controller;

import com.beauty.salon.dto.AuthRequestDTO;
import com.beauty.salon.dto.AuthResponseDTO;
import com.beauty.salon.security.JwtUtil;
import lombok.RequiredArgsConstructor;
import org.springframework.security.authentication.AuthenticationManager;
import org.springframework.security.authentication.UsernamePasswordAuthenticationToken;
import org.springframework.security.core.userdetails.UserDetails;
import org.springframework.security.core.userdetails.UserDetailsService;
import org.springframework.web.bind.annotation.*;

@RestController
@RequestMapping("/api/auth")
@RequiredArgsConstructor
public class AuthController {

```

```

private final AuthenticationManager authenticationManager;

private final JwtUtil jwtUtil;

private final UserDetailsService userDetailsService;

@PostMapping("/login")

public AuthResponseDTO login(@RequestBody AuthRequestDTO request) {

    authenticationManager.authenticate(

        new
UsernamePasswordAuthenticationToken(request.getEmail(),
request.getPassword())

    );

    UserDetails user =
userDetailsService.loadUserByUsername(request.getEmail());

    String token = jwtUtil.generateToken(user);

    return new AuthResponseDTO(token);

}

}

```

6. Авторизация и аутентификация

Безопасность приложения реализована с помощью Spring Security и JWT (JSON Web Token). Приложение работает в stateless-режиме — сессии не хранятся на сервере, каждый запрос проверяется по токену.

6.1. Роли и права доступа

В системе предусмотрены три роли:

 **Таблица доступа по ролям**

Эндпоинт	System Admin	Salon Admin	Cosmetologist
/api/clients/**	✓	✓	✗
/api/employees/**	✓	✗	✗
/api/procedures/**	✓	✓	✗
/api/appointments/**	✓	✓	✓
/api/schedules/**	✓	✓	✗

Рисунок 11-Роли и права

6.2. Конфигурация безопасности

В классе SecurityConfig настроены следующие правила:

- Публичные эндпоинты (доступны без токена): /api/auth/ и /actuator/health
- Доступ к /api/clients/ имеют роли System Administrator и Salon Administrator
- Доступ к /api/employees/ имеет только System Administrator
- Доступ к /api/procedures/ имеют роли System Administrator и Salon Administrator
- Доступ к /api/appointments/ имеют все три роли

- Доступ к `/api/schedules/` имеют роли `System Administrator` и `Salon Administrator`

CSRF защита отключена, так как используется JWT

Сессии не создаются (stateless режим)

6.3. Механизм JWT

JWT используется для аутентификации и авторизации пользователей. При успешном входе сервер генерирует токен, который клиент передаёт в заголовке `Authorization: Bearer <token>` при каждом запросе.

`JwtUtil` — класс, который отвечает за:

- Генерацию JWT-токена при успешном входе
- Извлечение email пользователя из токена
- Проверку срока действия токена
- Валидацию токена

`JwtAuthenticationFilter` — фильтр, который перехватывает каждый входящий запрос, извлекает токен из заголовка, проверяет его и устанавливает аутентификацию в контекст Spring Security.

6.4. Загрузка пользователя

`CustomUserDetailsService` — класс, который загружает пользователя из базы данных по email. Он находит сотрудника в таблице `employees`, определяет его роль через связную таблицу `employee_roles` и возвращает объект `UserDetails`, необходимый для работы Spring Security.

6.5. Демонстрация работы авторизации

1. Вход в систему (получение JWT-токена)

Запрос к `POST /api/auth/login` с email и паролем:

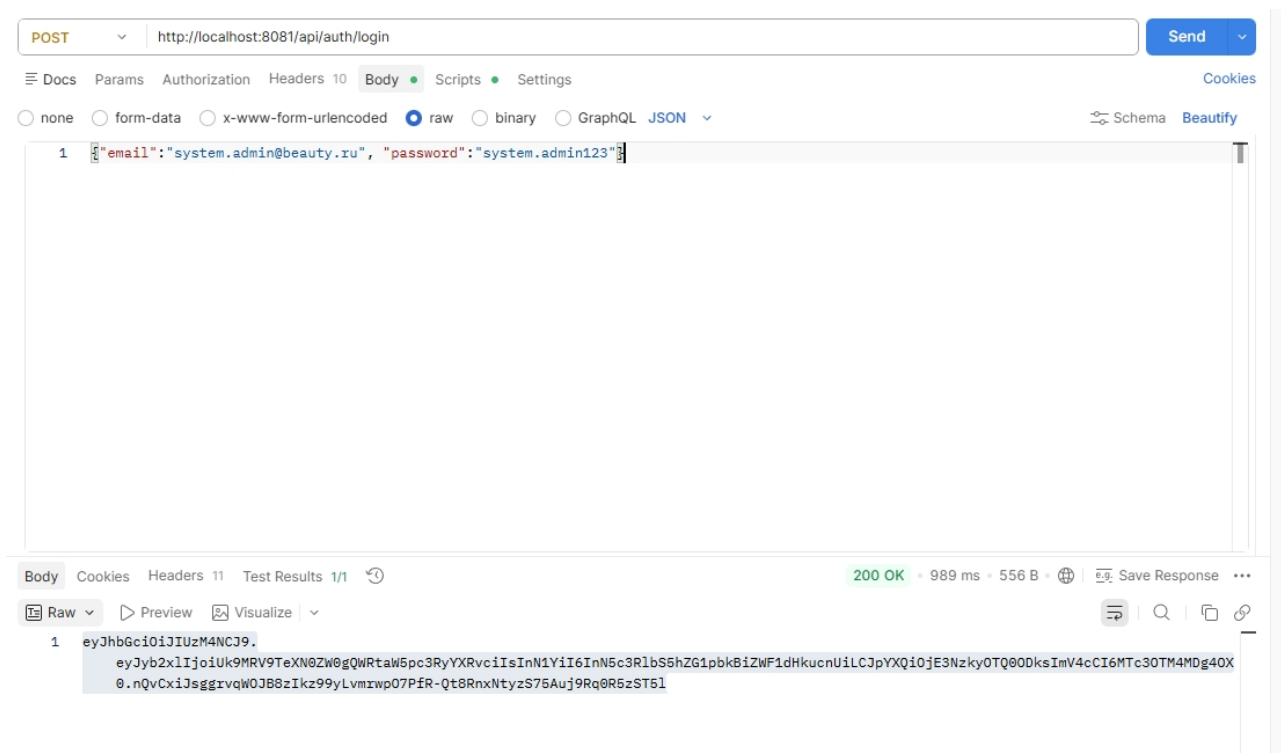


Рисунок 12-Вход в систему

2. Доступ к защищённому эндпоинту с токеном

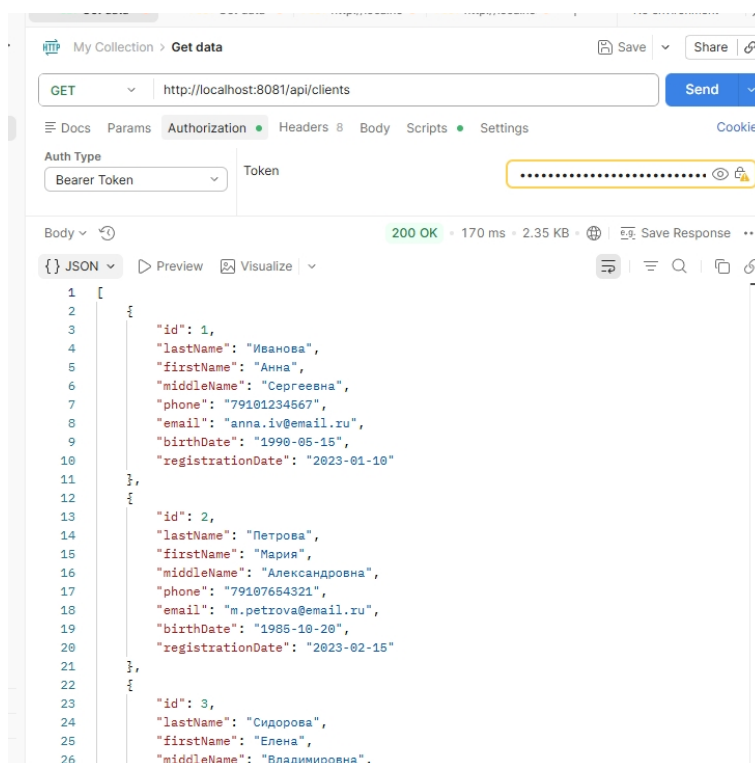


Рисунок 13- Доступ к защищённому эндпоинту с токеном

3. Отказ в доступе без токена

Попытка обратиться к защищённому эндпоинту без токена:

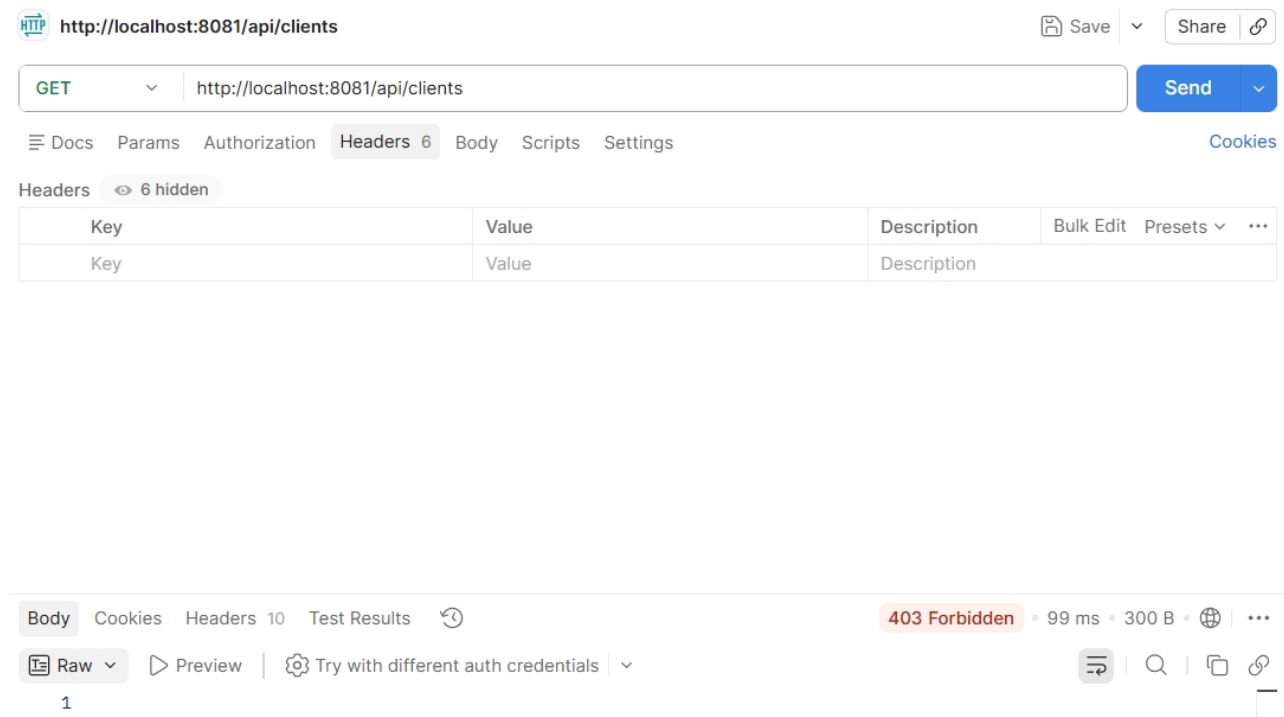


Рисунок 14-Отказ в доступе

7. Миграции базы данных (Liquibase)

Для управления структурой базы данных используется Liquibase. Все изменения описаны в файле `db.changelog-master.yaml`. При запуске приложения Liquibase автоматически создаёт таблицы и заполняет их начальными данными.

Файл миграции (фрагмент):

yaml

```
- changeSet:
  id: 3
  author: beauty-salon
  comment: "Create clients table"
  changes:
    - createTable:
```

```
tableName: clients
columns:
  - column:
      name: id
      type: INT
      autoIncrement: true
      constraints:
        primaryKey: true
        nullable: false
  - column:
      name: last_name
      type: VARCHAR(50)
      constraints:
        nullable: false
  - column:
      name: first_name
      type: VARCHAR(30)
      constraints:
        nullable: false
  - column:
      name: middle_name
      type: VARCHAR(30)
  - column:
      name: phone
      type: VARCHAR(11)
      constraints:
        nullable: false
        unique: true
```

- column:
 - name: email
 - type: VARCHAR(150)
 - constraints:
 - nullable: false
 - unique: true
- column:
 - name: birth_date
 - type: DATE
 - constraints:
 - nullable: false
- column:
 - name: registration_date
 - type: DATE
 - constraints:
 - nullable: false

Результат в базе данных (таблицы созданы):

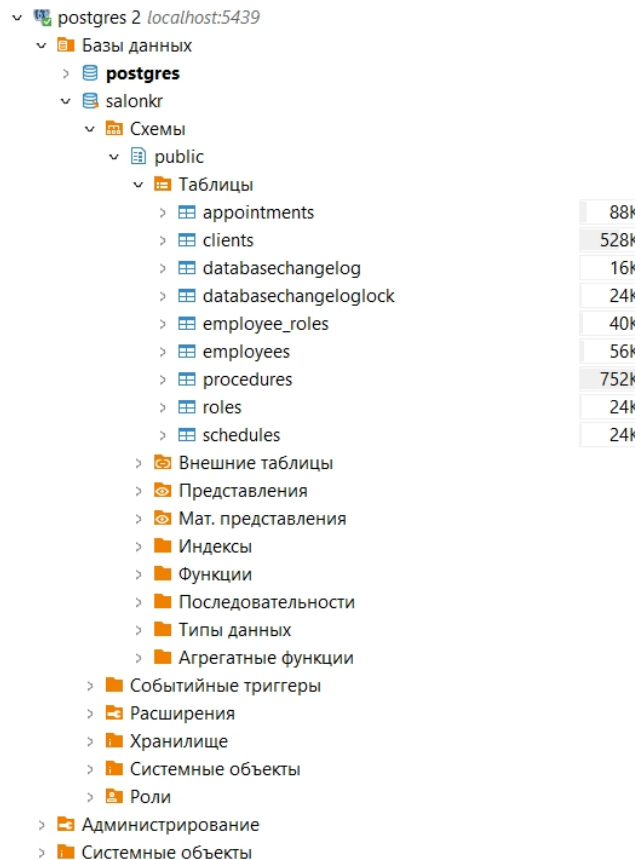


Рисунок 15-Созданные таблицы

Начальные данные в таблице Роли:

Свойства Данные Диаграмма				
roles Введите SQL выражение чтобы отфильтровать результаты				
Таблица		123 id	A-Z name	A-Z description
	1	1	Cosmetologist	Выполняет процедуры для клиентов.
	2	2	Salon Administrator	Управляет расписанием и записями клиентов.
	3	3	System Administrator	Управляет всеми аспектами системы.

Рисунок 16-Начальные данные в таблице Роли

8. Тестирование

В проекте реализованы unit- и интеграционные тесты. Для запуска тестов используется команда `./gradlew test`. Покрытие кода тестами проверяется с помощью плагина JaCoCo. Требование лабораторной работы — не менее 70%.

8.1. Запуск всех тестов

Результат выполнения команды `./gradlew test` (все тесты пройдены):

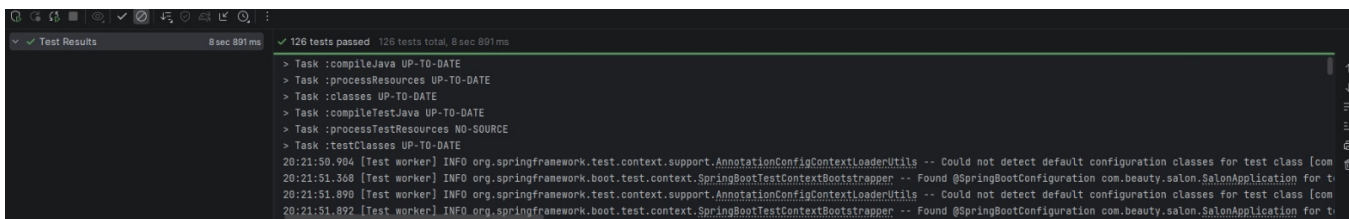


Рисунок 17-Результат работы тестов

8.2. Покрытие кода тестами (JaCoCo)

Отчёт JaCoCo с общим процентом покрытия (не менее 70%):

Element ^	Class, %	Method, %	Line, %	Branch, %
✓ com.beauty.salon	83% (15/18)	77% (71/92)	75% (263/3...	28% (13/46)
> config	100% (1/1)	100% (5/5)	100% (20/20)	100% (0/0)
> controller	100% (7/7)	100% (32/32)	100% (41/41)	100% (0/0)
> dto	100% (0/0)	100% (0/0)	100% (0/0)	100% (0/0)
> entity	100% (1/1)	14% (1/7)	14% (1/7)	100% (0/0)
> repository	100% (0/0)	100% (0/0)	100% (0/0)	100% (0/0)
> security	33% (1/3)	9% (1/11)	26% (14/52)	0% (0/14)
> service	100% (5/5)	88% (32/36)	82% (187/2...	40% (13/32)
SalonApplication	0% (0/1)	0% (0/1)	0% (0/2)	100% (0/0)

Рисунок 18-Покрытие кода тестами

9. Логирование и Actuator

Для мониторинга работы приложения и отслеживания событий используются логирование и Spring Boot Actuator.

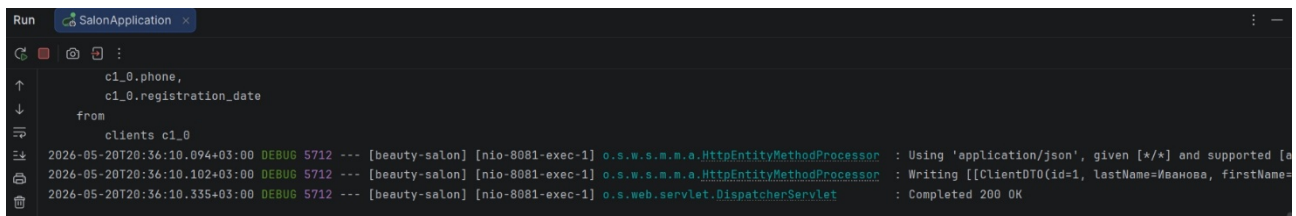
9.1. Логирование

Логирование настроено в файле application.properties:

Таблица 1-Логирование

Уровень	Пакет / класс	Описание
DEBUG	com.beauty.salon	Логи приложения
DEBUG	org.hibernate.SQL	SQL-запросы к базе данных
DEBUG	org.springframework.web	HTTP-запросы и ответы

Пример логов при работе приложения (консоль):



```
Run SalonApplication x
c1_0.phone,
c1_0.registration_date
from
clients c1_0
2026-05-20T20:36:10.094+03:00 DEBUG 5712 --- [beauty-salon] [nio-8081-exec-1] o.s.w.s.m.a.HttpEntityMethodProcessor : Using 'application/json', given [*/] and supported [
2026-05-20T20:36:10.102+03:00 DEBUG 5712 --- [beauty-salon] [nio-8081-exec-1] o.s.w.s.m.a.HttpEntityMethodProcessor : Writing [[ClientDTO(id=1, lastName=Иванова, firstName=
2026-05-20T20:36:10.335+03:00 DEBUG 5712 --- [beauty-salon] [nio-8081-exec-1] o.s.web.servlet.DispatcherServlet : Completed 200 OK
```

Рисунок 19-Пример логов при работе приложен

9.2. Spring Boot Actuator

Actuator предоставляет эндпоинты для мониторинга состояния приложения. В application.properties подключены следующие эндпоинты:

properties

management.endpoints.web.exposure.include=health,info,metrics

management.endpoint.health.show-details=always

Доступные эндпоинты:

Таблица 2-Эндпоинты

Эндпоинт	Описание
/actuator/health	Статус приложения (здорово / не здорово)
/actuator/info	Информация о приложении
/actuator/metrics	Метрики (память, процессор, количество запросов)

Проверка здоровья приложения:

```
Автоформатировать ☒
{
  "components": {
    "db": {
      "details": {
        "database": "PostgreSQL",
        "validationQuery": "isValid()"
      },
      "status": "UP"
    },
    "diskSpace": {
      "details": {
        "total": 256058060800,
        "free": 143116193792,
        "threshold": 10485760,
        "path": "C:\\Users\\Admin\\Downloads\\salon (1)\\salon\\.\\",
        "exists": true
      },
      "status": "UP"
    },
    "livenessState": {
      "status": "UP"
    },
    "ping": {
      "status": "UP"
    },
    "readinessState": {
      "status": "UP"
    },
    "ssl": {
      "details": {
        "expiringChains": [],
        "invalidChains": [],
        "validChains": []
      },
      "status": "UP"
    }
  },
  "groups": [
    "liveness",
    "readiness"
  ],
  "status": "UP"
}
```

Рисунок 20-Проверка здоровья приложения

Метрики приложения (пример):

Полный список метрик

```
{
```

```
"names": [  
  "application.ready.time",  
  "application.started.time",  
  "disk.free",  
  "disk.total",  
  "executor.active",  
  "executor.completed",  
  "executor.pool.core",  
  "executor.pool.max",  
  "executor.pool.size",  
  "executor.queue.remaining",  
  "executor.queued",  
  "hikaricp.connections",  
  "hikaricp.connections.acquire",  
  "hikaricp.connections.active",  
  "hikaricp.connections.creation",  
  "hikaricp.connections.idle",  
  "hikaricp.connections.max",  
  "hikaricp.connections.min",  
  "hikaricp.connections.pending",  
  "hikaricp.connections.timeout",  
  "hikaricp.connections.usage",  
  "http.server.requests",  
  "http.server.requests.active",  
  "jdbc.connections.active",  
  "jdbc.connections.idle",  
  "jdbc.connections.max",  
  "jdbc.connections.min",  
  "jvm.buffer.count",  
  "jvm.buffer.memory.used",  
  "jvm.buffer.total.capacity",  
  "jvm.classes.loaded",  
  "jvm.classes.loaded.count",  
  "jvm.classes.unloaded",  
  "jvm.compilation.time",  
  "jvm.gc.live.data.size",  
  "jvm.gc.max.data.size",  
  "jvm.gc.memory.allocated",  
  "jvm.gc.memory.promoted",  
  "jvm.gc.overhead",  
  "jvm.info",  
  "jvm.memory.committed",  
  "jvm.memory.max",  
  "jvm.memory.usage.after.gc",
```



```
"jvm.memory.used",
"jvm.threads.daemon",
"jvm.threads.live",
"jvm.threads.peak",
"jvm.threads.started",
"jvm.threads.states",
"logback.events",
"process.cpu.time",
"process.cpu.usage",
"process.start.time",
"process.uptime",
"spring.data.repository.invocations",
"spring.security.authorizations",
"spring.security.authorizations.active",
"spring.security.filterchains",
"spring.security.filterchains.JwtAuthenticationFilter.after",
"spring.security.filterchains.JwtAuthenticationFilter.before",
"spring.security.filterchains.access.exceptions.after",
"spring.security.filterchains.access.exceptions.before",
"spring.security.filterchains.active",
"spring.security.filterchains.authentication.anonymous.after",
"spring.security.filterchains.authentication.anonymous.before",
"spring.security.filterchains.authorization.after",
"spring.security.filterchains.authorization.before",
"spring.security.filterchains.context.async.after",
"spring.security.filterchains.context.async.before",
"spring.security.filterchains.context.holder.after",
"spring.security.filterchains.context.holder.before",
"spring.security.filterchains.context.servlet.after",
"spring.security.filterchains.context.servlet.before",
"spring.security.filterchains.header.after",
"spring.security.filterchains.header.before",
"spring.security.filterchains.logout.after",
"spring.security.filterchains.logout.before",
"spring.security.filterchains.requestcache.after",
"spring.security.filterchains.requestcache.before",
"spring.security.filterchains.session.management.after",
"spring.security.filterchains.session.management.before",
"spring.security.filterchains.session.urlencoding.after",
"spring.security.filterchains.session.urlencoding.before",
"spring.security.http.secured.requests",
"spring.security.http.secured.requests.active",
"system.cpu.count",
"system.cpu.usage",
```

```

"tomcat.sessions.active.current",
"tomcat.sessions.active.max",
"tomcat.sessions.alive.max",
"tomcat.sessions.created",
"tomcat.sessions.expired",
"tomcat.sessions.rejected"
]
}

```

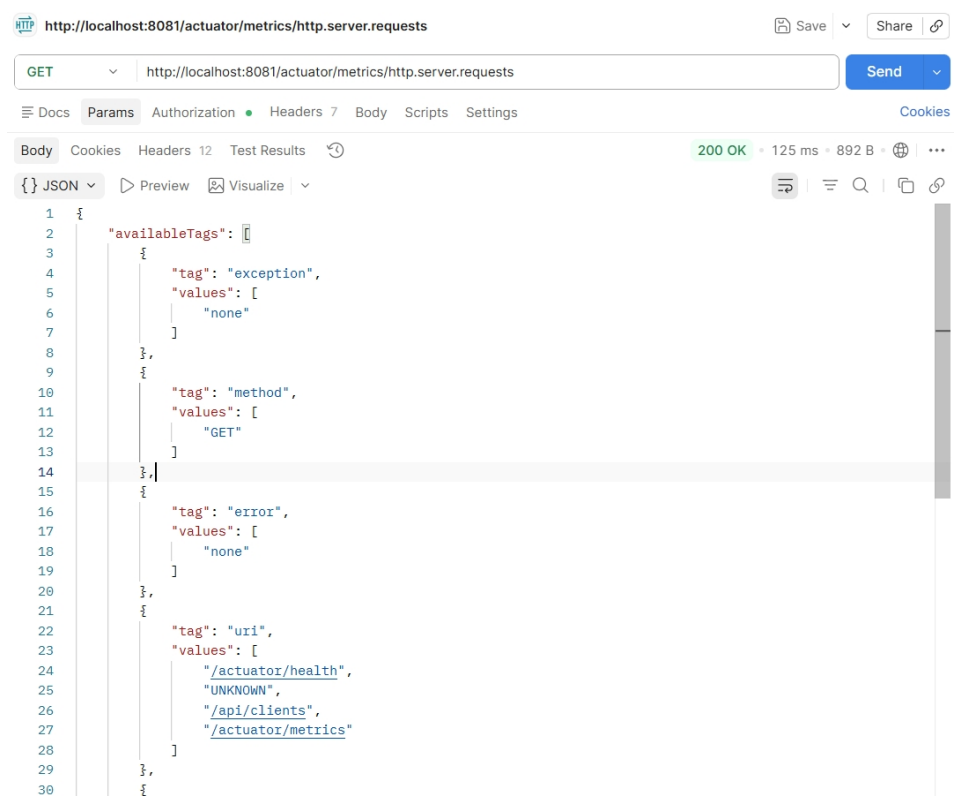


Рисунок 21-Пример метрики

```

Пример метрики http.server.requests: {
  "availableTags": [
    {
      "tag": "exception",
      "values": [
        "none"
      ]
    },
    {
      "tag": "method",
      "values": [
        "GET"
      ]
    },
    {
      "tag": "error",

```

```

"values": [
  "none"
],
{
  "tag": "uri",
  "values": [
    "/actuator/health",
    "UNKNOWN",
    "/api/clients",
    "/actuator/metrics"
  ],
},
{
  "tag": "outcome",
  "values": [
    "CLIENT_ERROR",
    "SUCCESS"
  ],
},
{
  "tag": "status",
  "values": [
    "200",
    "403"
  ],
},
],
"baseUnit": "seconds",
"measurements": [
  {
    "statistic": "COUNT",
    "value": 4.0
  },
  {
    "statistic": "TOTAL_TIME",
    "value": 2.9320246
  },
  {
    "statistic": "MAX",
    "value": 0.0780593
  },
],

```

```
"name": "http.server.requests"  
}
```

10. Вывод

В ходе выполнения лабораторной работы разработано веб-приложение «Beauty Salon» для автоматизации работы салона красоты.

В приложении реализовано семь сущностей (Client, Employee, Role, EmployeeRole, Procedure, Appointment, Schedule) с JPA-связями между ними. Для всех сущностей созданы CRUD-контроллеры, обеспечивающие операции создания, чтения, обновления и удаления.

Безопасность приложения построена на Spring Security и JWT. В системе реализованы три роли: System Administrator (полный доступ), Salon Administrator (управление клиентами, услугами, записью и расписанием) и Cosmetologist (просмотр своего расписания и записей).

Для управления схемой базы данных использован Liquibase. Миграции содержат создание всех таблиц и заполнение их начальными данными (роли, сотрудники, клиенты, услуги, расписание, записи).

Написаны unit- и интеграционные тесты. Покрытие кода тестами, проверенное с помощью JaCoCo, составляет не менее 70%, что соответствует требованию лабораторной работы.

Настроено логирование запросов (уровень DEBUG для SQL, HTTP и пакета приложения) и подключен Spring Boot Actuator с эндпоинтами health, info и metrics для мониторинга состояния приложения.

Все требования лабораторной работы выполнены. Приложение готово к внедрению и может использоваться для автоматизации записи клиентов и управления сотрудниками в салоне красоты.